DAY 06 / 15 · PHASE 2 — ETHEREUM & SOLIDITY

Solidity fundamentals

Tuesday 20 October 2026 · Manish Khadka · github.com/Khadka1996/Learning-Blockchain/tree/main/day-06

Learning objectives

1. Explain state vs local variables and where data lives: `storage`, `memory`, `calldata`.
2. Use mappings, structs, enums, arrays, constants and immutables, and know what each costs.
3. Control access with visibility, `view`/`pure`, `payable` and modifiers.
4. Fail loudly with custom errors; emit events in an order that apps can replay.
5. Send ETH safely with checks → effects → interaction plus a reentrancy guard, and prove it with an attacker contract.
6. Test like an attacker: boundaries, control experiments, random tests against a model, and mutation testing.

The concept, step by step

Step 1 — A contract is a program with its own bank account and memory

Its **state variables** sit in storage, the 32-byte lockers from Day 05, and survive between calls. **Local variables** exist only while a function runs. A `constructor` runs once at deployment. An `immutable` (like `owner`) is set there and copied into the bytecode, so it uses no storage slot.

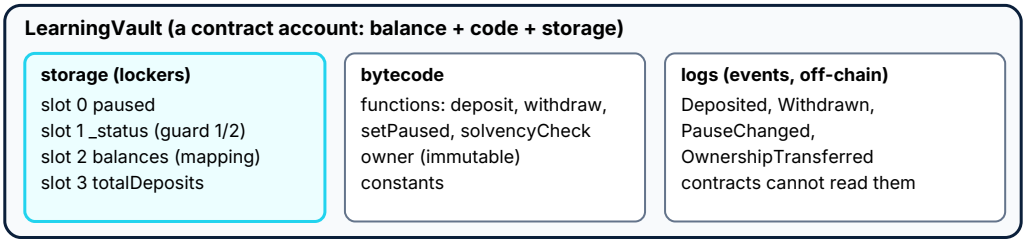


Figure 1 — Where each part of LearningVault lives. Storage is read raw with `eth_getStorageAt` in the tests.

Step 2 — Types, and where data lives

Value types (`uint256`, `bool`, `address`, `enum`) are copied. **Reference types** (arrays, structs, `string`) need a location: `storage` is the permanent on-chain data (a `storage` pointer edits the real thing), `memory` a temporary copy, `calldata` read-only call input and the cheapest for external functions. A `mapping` is a lookup table you cannot list. `StudyLog.sol` shows all of these in one small contract. Names are measured in **bytes**: 22 characters of "€" are 66 bytes and fail a 64-byte limit.

Step 3 — Who may call what

`external`/`public` decide who can call; `view` promises not to change state, `pure` not even to read it; `payable` lets ETH in. **Modifiers** wrap a function in reusable checks: `onlyOwner`, `whenNotPaused`, `nonReentrant`. They save repetition but do not replace testing every caller.

Step 4 — Fail loudly, and keep an honest diary

A `revert` undoes everything the call did. **Custom errors** such as `InsufficientBalance(available, requested)` carry data and cost less than text. **Events** are the public diary apps read; their *order* matters because apps replay them.

Step 5 — Sending ETH hands control to someone else's code

The receiver's `receive()` runs in the middle of your function. If your books still say "balance 1" it can ask again. Order: **check → update your state → send**, plus a guard as a second lock.

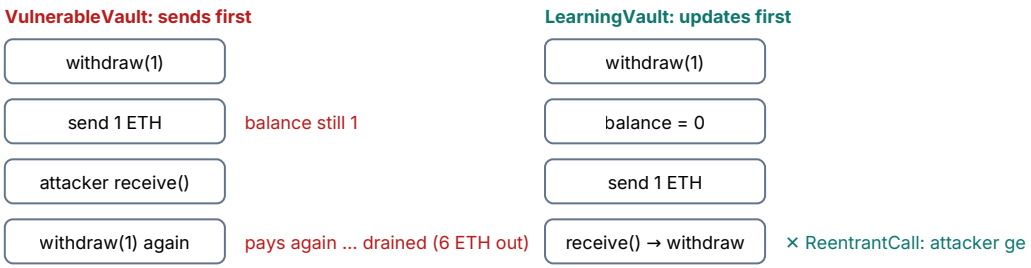


Figure 2 — The same attacker contract against both vaults (measured in `npm start`).

Key code walkthrough

The safe withdrawal (`contracts/LearningVault.sol`)

```
function withdraw(uint256 amount) external override nonReentrant {
    if (amount == 0) revert ZeroAmount();                                  // 1 checks
    uint256 available = balances[msg.sender];
    if (amount > available) revert InsufficientBalance(available, amount);

    uint256 remaining = available - amount;                                // 2 effects
    balances[msg.sender] = remaining;
    totalDeposits -= amount;
    emit Withdrawn(msg.sender, amount, remaining);                         //   event BEFORE the
call

    (bool success, ) = payable(msg.sender).call{value: amount}("");        // 3 interaction
    if (!success) revert TransferFailed();
}
```

The guard flips 1 ↔ 2, never to 0

```
modifier nonReentrant() {
    if (_status == _ENTERED) revert ReentrantCall();   // _ENTERED = 2
    _status = _ENTERED;  _;  _status = _NOT_ENTERED;    // _NOT_ENTERED = 1: slot never empty
}
```

A slot that goes 0 → 1 → 0 pays a fresh write each call (Day 05): 52,031 gas per withdrawal vs 42,859 with 1 ↔ 2.

Results from the run

Run	Result
`npm start` deploy	376,377 gas; 1,385 bytes of runtime code; `owner` is in the bytecode, not in a slot
Deposits	first 69,667 gas (0 → value), second 35,467 gas (value → value)
Reentrancy	VulnerableVault: attacker ends with 6.0 ETH after 5 re-entries, vault holds 0. LearningVault: `ReentrantCall`, attacker gets its own 1 ETH, vault keeps 5
Failed payout	Recipient with no `receive()` → `TransferFailed`, recorded balance unchanged
Pause	Deposits → `ContractPaused`; withdrawals still work; non-owner → `NotOwner`
Quality	75 / 75 tests · ESLint + Solhint clean · `npm audit --omit=dev` 0 · Slither: no real issue · mutation testing 16 / 16 caught

How I tested it

1. A control before a claim

"The attack fails" means nothing if the attacker could not win anywhere. The tests first drain a deliberately broken `VulnerableVault` (6 ETH out), then show `LearningVault` refusing the same attacker with `ReentrantCall`. The same idea checks the enum: a hand-built call with value 2 works, value 3 is refused.

2. Random operations against a model

200 random programs of deposits, withdrawals and pause toggles run on the contract and on a 15-line JavaScript model. After every step both must agree on balances, `totalDeposits`, the contract's real ETH and on which custom error a failing call returns. (fast-check, fixed seed, so a failure is reproducible.)

3. Break it on purpose

`npm run qa:mutation` changes the contract one line at a time (remove the guard, send before updating, `>` to `>=`, drop an event …) and runs every test. 16 broken versions, 16 caught. The first 19 tests caught only 10 of 15 equivalent breaks: green did not mean watching.

```
$ npm run qa:mutation
KILLED  (1 test failed)   reentrancy guard removed
KILLED  (2 tests failed)  interaction before effects (guard kept)
KILLED  (1 test failed)   failed payout is ignored
...
16 of 16 mutants killed.
```

QA & security findings

The morning's 19 green tests let **5 of 15** broken versions of the contract through. Evening review found 6 issues (full table in `QA.md`).

ID	Sev.	Problem	Fix
M1	**Medium**	Pause blocked `withdraw` too. With an immutable owner, a lost or malicious key locked all ETH forever.	Pause stops deposits only; exits always open.
L1	**Low**	`Withdrawn` emitted after the call: a re-depositing receiver made the log read *Deposited, Deposited(newBalance 50), Withdrawn*, a history that never happened.	Emit with the other effects, before the call.
L2	**Low**	Guard 0 → 1 → 0 paid a new storage write each call: 52,031 gas.	Guard 1 ↔ 2: 42,859 gas (−17.6 %).
L3	**Low**	A constructor test that could never fail; reentrancy guard, failed payout and forced ETH untested.	Real assertions, attacker + control vault, 16 mutants all caught.
I1, I2	**Info**	Dead `ZeroAddress` error and a wrong comment; a hand-rolled lock in `VaultPlus` flagged by Slither.	Removed / modifier used.

Accepted: not independently audited; the owner can stop new deposits; a pause cannot stop an exploit that goes through `withdraw`. Do not deploy with real funds.

Common mistakes

- Sending ETH first and updating the balance after: the classic reentrancy bug.
- Trusting a green test that "the attack fails" without first showing the attack works on a weak target.
- A pause that also freezes withdrawals: an owner-key problem becomes a user-funds problem.
- Emitting an event after an external call; apps that replay logs see events out of order.
- Assuming `address(this).balance` equals your tracked deposits: anyone can force ETH in with `selfdestruct`.
- Looping over a list anyone can grow inside a state-changing function (denial of service).
- Counting characters instead of bytes in a length limit.

Interview questions

1. What is checks-effects-interactions and why does it matter?

Validate, then update your own state, then make external calls. During the call the other side's code runs; if your state is already final it cannot exploit a stale balance.

2. Why use both a guard and the ordering?

Two independent layers: remove either and the other still stops the drain. Mutation tests check each layer separately.

3. `immutable` vs `constant` vs a normal variable?

`constant` is fixed at compile time, `immutable` at deployment; both live in the bytecode, so reading them is cheap. A normal variable occupies a storage slot.

4. Why are custom errors preferred over `require` with text?

Cheaper to deploy and to revert, and they carry typed data that tests and apps can decode.

5. When is pausing withdrawals justified?

To stop a live drain, but it freezes honest users and, if the key is lost while paused, forever. Safer designs add a timelock, a multisig or an auto-expiring pause.

Exercises summary

Level	Task	Result of the solution
Easy	`depositCount`, `getBalance`, immutable vs mutable	3 deposits → count 3; failed deposits are not counted
Medium	Minimum deposit error, indexed-event test, reverted withdrawal changes nothing	`BelowMinimum(999, 1000)`; Deposited log has 2 topics
Challenge	Attacker contract, pause trade-off, two-step ownership	Drains the broken vault (6 ETH), refused by the real one; a typo address can never take over

Cheat sheet

`external / public`	who can call · `view` reads · `pure` reads nothing · `payable` accepts ETH
`revert Name(args)`	undo everything, with typed data · `emit Event(...)` writes a log
mapping slot	`keccak256(abi.encode(key, slot))`
ETH send	`(bool ok,) = to.call{value: v}(""); if (!ok) revert …;` after updating state
commands	`npm start · test · lint · slither · qa:mutation · ex:easy|medium|challenge`

Key terms

Term	Plain meaning
State variable	A value kept in on-chain storage between calls
Modifier	A reusable check wrapped around a function
Custom error	A named revert reason carrying data
Event	A log entry apps can read; `indexed` arguments are searchable
Interface	Function signatures with no code; implementers must provide them
Reentrancy	A paid contract calls back before you finished updating your books
Mutation testing	Break the code on purpose; a good test suite fails every time
Forced ETH	ETH pushed into a contract by `selfdestruct` even if it refuses payments

What I'd explore next

- Day 07: ERC-20 tokens, allowances and OpenZeppelin, with fuzz tests.
- Day 09: reentrancy and access-control exploits in depth, and a timelock or multisig owner.
- Day 11: custom-error and storage-packing gas savings; upgradeable proxies.

References

1. Solidity docs 0.8.24: Structure of a Contract, Contracts, Types. https://docs.soliditylang.org/en/v0.8.24/
2. Solidity docs: Security considerations (reentrancy). https://docs.soliditylang.org/en/v0.8.24/security-considerations.html
3. SWC-107: Reentrancy. https://swcregistry.io/docs/SWC-107
4. OpenZeppelin ReentrancyGuard (why 1 and 2). https://docs.openzeppelin.com/contracts/5.x/api/utils#ReentrancyGuard
5. Hardhat documentation. https://hardhat.org/docs · ethers.js v6. https://docs.ethers.org/v6/
6. Slither static analyzer. https://github.com/crytic/slither
7. EIP-2929 / EIP-3529 (storage gas prices, from Day 05). https://eips.ethereum.org/EIPS/eip-2929